

AWS EKS Storage Master Deep Dive

This document combines the complete practical lab, troubleshooting, architecture analysis, Kubernetes CSI internals, EFS vs S3 behavioral testing, Pod Identity configuration, production design decisions, and real-world lessons learned while implementing Amazon EFS CSI Driver and Mountpoint for Amazon S3 CSI Driver on EKS.

Topics Covered

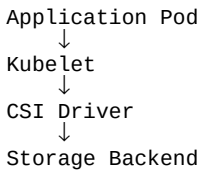
- Kubernetes CSI Architecture
- EFS Internal Architecture
- S3 Mountpoint CSI Internal Architecture
- IRSA vs Pod Identity
- EFS Access Points
- EFS Mount Targets
- RWX Storage
- Private VPC Endpoints
- S3 Object Semantics
- POSIX Filesystem Semantics
- Concurrent Writes
- File Locking
- Sequential Read Optimization
- Real Production Design Patterns
- Real Troubleshooting Performed

SECTION 1 - Understanding Kubernetes Storage Architecture

Most Kubernetes engineers learn storage at the YAML level only. But real production understanding begins when you understand what actually happens inside the node during volume mounts.

Container Storage Interface (CSI)

CSI is the standard interface between Kubernetes and external storage systems. Instead of Kubernetes implementing every storage backend internally, storage vendors provide CSI drivers.



CSI drivers usually contain: Controller component Node component Provisioner sidecars Mount helpers

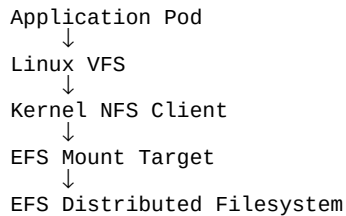
Controller vs Node Plugin

Component	Purpose
Controller	Provisioning, API coordination, volume lifecycle
Node Plugin	Actual mount operations on worker nodes

SECTION 2 - Amazon EFS Deep Dive

What is EFS?

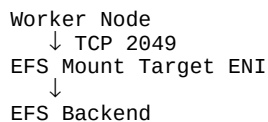
Amazon EFS is a managed distributed POSIX filesystem built on NFSv4. It behaves like a real Linux shared filesystem. Multiple nodes and multiple pods can simultaneously mount the same filesystem.



EFS Mount Targets

Mount targets are ENIs created inside subnets. Each Availability Zone requires its own mount target. Your worker nodes communicate with these mount targets using NFS protocol on TCP 2049.

Why mount targets need: Subnets → because ENIs live inside VPC subnets Security Groups → because NFS traffic must be controlled



EFS Access Points

Access Points provide application-specific entry points into EFS. They solve multi-tenant filesystem problems.

Feature	Purpose
Root Directory	Restrict application view into filesystem
POSIX User	Force UID/GID
Creation Info	Auto-create directory with ownership

Important discovery: If POSIX user is NOT configured, the application's own UID/GID is used.

Real Terraform Used

```
resource "aws_efs_file_system" "this" {
  encrypted          = true
  throughput_mode    = "elastic"
}

resource "aws_efs_mount_target" "this" {
  for_each = toset(var.private_subnet_ids)

  file_system_id = aws_efs_file_system.this.id
  subnet_id      = each.value
}

resource "aws_efs_access_point" "this" {
  file_system_id = aws_efs_file_system.this.id

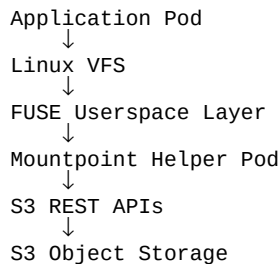
  root_directory {
    path = "/k8s"
  }
}
```

}
}

SECTION 3 - Mountpoint for Amazon S3 CSI Deep Dive

Critical Understanding

S3 is NOT a filesystem. S3 is object storage. Mountpoint for S3 only EMULATES filesystem behavior.



This is the single most important concept in this entire lab.

Why S3 Behaves Differently

Filesystem World	Object Storage World
Inodes	Objects
Byte modifications	Whole object replacement
File locks	No lock semantics
append()	PUT object
rename()	Copy + Delete

Internal Mountpoint Pods

One of the biggest discoveries during troubleshooting was: the S3 CSI driver dynamically creates helper pods inside the mount-s3 namespace.

```
kubectl get pods -n mount-s3
```

These helper pods perform: FUSE operations S3 API communication Credential usage Mount lifecycle management

SECTION 4 - IRSA vs Pod Identity

IRSA

IAM Roles for Service Accounts uses an OIDC provider. ServiceAccounts are annotated with IAM roles. Pods assume roles using AssumeRoleWithWebIdentity.

```
eks.amazonaws.com/role-arn
```

Pod Identity

EKS Pod Identity is AWS's newer mechanism. It avoids OIDC complexity. Credentials are injected dynamically.

```
AWS_CONTAINER_CREDENTIALS_FULL_URI
AWS_CONTAINER_AUTHORIZATION_TOKEN_FILE
```

Major lesson learned: Application Pod Identity and CSI Driver Identity are DIFFERENT concerns.

SECTION 5 - Real Troubleshooting Journey

Issue 1 - EFS PVC Pending

Observed: PVC remained Pending. Pods could not schedule.

Root Cause: Incorrect FileSystemId inside StorageClass.

```
kubectl describe pvc efs-pvc
```

Fix: Verified EFS exists Corrected StorageClass Restarted CSI controller

Issue 2 - S3 No Signing Credentials

Observed error:

```
No signing credentials available
```

Root Cause: Mountpoint helper pods in namespace mount-s3 were using default ServiceAccount without Pod Identity.

Important architectural discovery: Application pod credentials were irrelevant because the actual S3 API calls were being made by helper mountpoint pods.

```
aws eks create-pod-identity-association --namespace mount-s3 --service-account default
```

Issue 3 - Git Bash Path Rewriting

Git Bash on Windows rewrote:

```
/data/file.txt
↓
C:/Program Files/Git/data/file.txt
```

Fix: Always use:

```
kubectl exec -- sh -c 'cat /data/file.txt'
```

SECTION 6 - Real Feature Testing

1. File Locking Test

EFS: flock() worked perfectly. Second pod blocked until lock released.

S3: Failed with:

Operation not permitted

Why? S3 has no inode locking semantics.

2. Append Test

EFS: append worked correctly.

S3: append failed because object storage cannot safely mutate existing object bytes.

3. Random Byte Write Test

EFS supported:

```
dd seek=100
```

S3 failed because objects are immutable blobs.

4. chmod and Symlink Test

EFS: Real POSIX behavior.

S3: Failed or partially emulated because S3 has no inode graph.

5. Concurrent Writes

EFS: Stable concurrent writes.

S3: Unsafe concurrent mutations. Last writer wins behavior possible.

SECTION 7 - Performance Observations

Test	EFS	S3 Mountpoint
Small file latency	~0.00s	~0.12s
100MB sequential read	~0.24s	~0.68s
Concurrent writes	Good	Unsafe
append()	Supported	Not Supported

Interesting insight: Mountpoint for S3 is optimized for HIGH THROUGHPUT SEQUENTIAL READS, not mutable POSIX workloads.

SECTION 8 - Real Production Design Decisions

Use EFS When

Applications expect Linux filesystem semantics
Jenkins Home Shared uploads
RWX Kubernetes workloads
Applications use flock()
Concurrent writers exist
POSIX apps

Use S3 When

Immutable objects Backups Static assets AI datasets Data lakes Artifact storage Media streaming Sequential reads

Never Use S3 Mountpoint For

Databases SQLite Shared mutable logs Lock-heavy applications append() dependent applications

SECTION 9 - Kubernetes Logging Lessons

One extremely important lesson: Do NOT share active writable log files across replicas.

Modern Kubernetes logging:

```
Application
  ↓ stdout/stderr
Container Runtime
  ↓
FluentBit / Fluentd
  ↓
CloudWatch / Loki / ELK / S3
```

S3 is excellent for immutable archived logs, NOT shared active append-only files.

SECTION 10 - Final Master Conclusion

The biggest lesson from this entire lab:

```
Mounting S3 does NOT magically convert object storage
into a real filesystem.
```

Mountpoint for S3 only EMULATES filesystem behavior. That explains: why flock() fails why append() fails why concurrent writes are unsafe why random writes fail why databases break

EFS behaves like Linux because it IS a distributed filesystem. S3 behaves differently because it is fundamentally object storage.